

MODULE :2

Process Management and Threads

2.1 Process Concept

- **History**

1. Early computers executes a batch systems.
 - These systems ran one job at a time.
 - A job was a complete task sent to the computer—like printing or calculations.
2. Then came time-sharing systems.
 - These allowed many users to run programs at the same time.
 - Each user's program was often called a task.

- **Process**

- A Process is a program under execution.
- Its current activity is indicated by Program Counter (PC) and the content of the processor's register.

2.1.1 The Process

Process memory is divided into four sections as shown in the figure below:

- The **Stack** is used to store temporary data such as local variables, function parameters, function return values, return address etc.
- The **Heap** which is memory that is dynamically allocated during process run time
- The **Data** section stores global variables.
- The **Text** section comprises the compiled program code.
- **Note*** there is a free space between the stack and the heap. When the stack is full, it grows downwards and when the heap is full, it grows upward

Process State

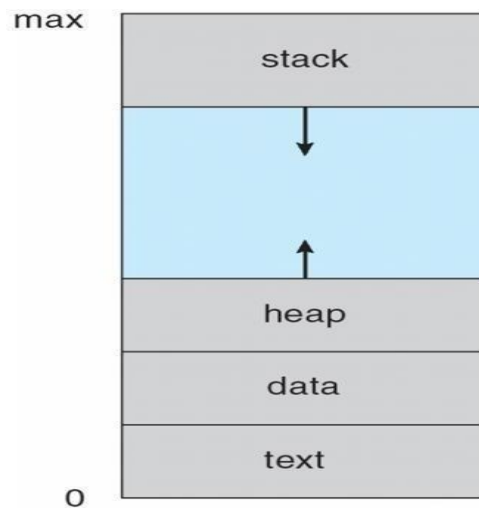


Figure 2.1: Process in Memory

2.1.2 Process State: A Process has 5 states. Each process may be in one of the following states

1. **New** - The process is in the stage of being created.
2. **Ready** - The process has all the resources it needs to run. It is waiting to be assigned to the processor.
3. **Running** – Instructions are being executed.
4. **Waiting** - The process is waiting for some event to occur. For example, the process may be waiting for keyboard input, disk access request, inter-process messages, a timer to go off, or a child process to finish.
5. **Terminated** - The process has completed its execution.

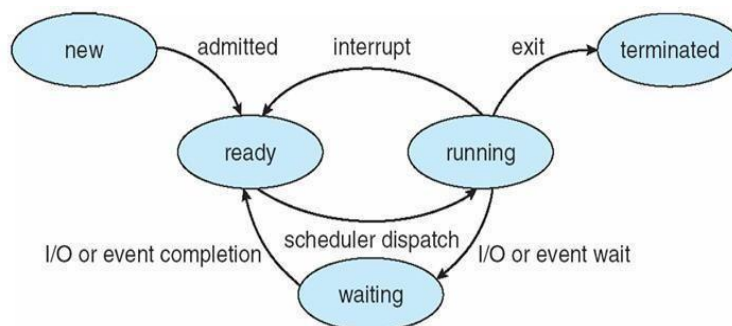


Figure 2.2: Diagram of process state

Note* A process is a program that is running or waiting to run on a computer.

- At any moment, only one process can actually run on a single processor core.
- But, many other processes can be ready and waiting their turn to run.

2.1.3 Process Control Block

For each process there is a Process Control Block (PCB), which stores the process-specific information as shown below –

- **Process State** – The state of the process may be new, ready, running, waiting, and so on.
- **Program counter** – The counter indicates the address of the next instruction to be executed for this process.
- **CPU registers** - The registers vary in number and type, depending on the computer architecture. They include accumulators, index registers, stack pointers, and general-purpose registers. Along with the program counter, this state information must be saved when an interrupt occurs, to allow the process to be continued correctly afterward.
- **CPU scheduling information**- This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.
- **Memory-management information** – This includes information such as the value of the base and limit registers, the page tables, or the segment tables.
- **Accounting information** – This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.
- **I/O status information** – This information includes the list of I/O devices allocated to the process, a list of open files, and so on.

The PCB simply serves as the repository for any information that may vary from process to process.



Figure 2.3: Process control block (PCB)

2.2 Process Scheduling

Process scheduling works in Linux and its goals are two main concepts in operating systems: **multiprogramming** and **time-sharing**.

- **Multiprogramming** aims to keep the CPU busy all the time by having multiple processes loaded and ready to run. If one process waits (say for input/output), another can use the CPU, maximizing CPU utilization.
- **Time-sharing** focuses on giving the user the illusion that many processes run at once by switching the CPU between processes very quickly. This allows users to interact with programs as if they were running simultaneously.

To achieve these two objectives, we need **Process Scheduling**.

Process Scheduler selects among available processes for next execution on CPU.

It maintains Scheduling Queue of Processes.

- Job Queue: List of all processes in the system.
- Ready Queue: Set of all processes residing in main memory, ready and waiting to execute.
- Device Queue: Set of processes waiting for an I/O device.



I/O-bound process – spends more time doing I/O than computations, CPU-bound process – spends more time doing computations and few I/O operations. An efficient scheduling system will select a good mix of **CPU-bound** processes and **I/O bound** processes.

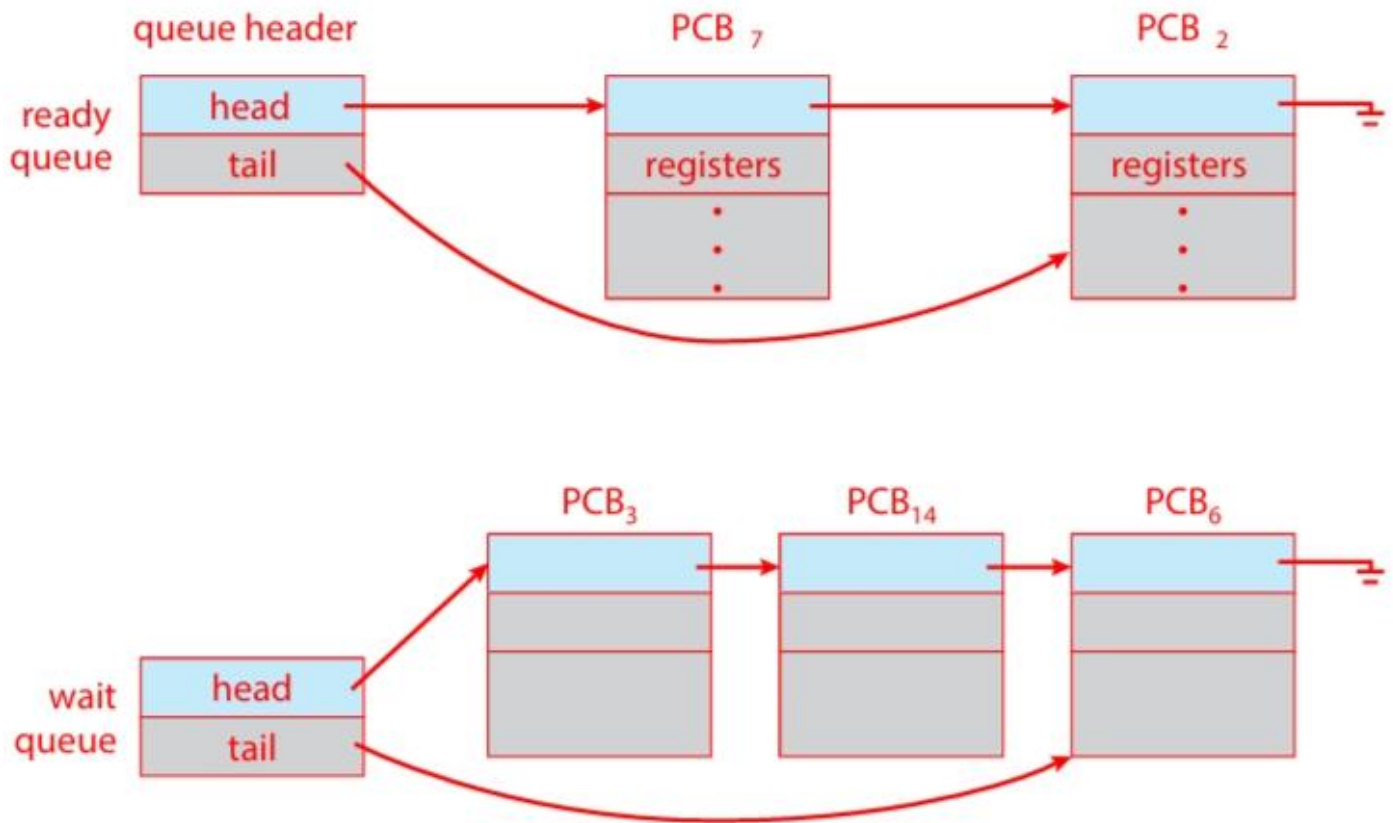


Figure 2.6: The ready queue and wait Queue

- If the scheduler selects **more I/O bound process**, then I/O queue will be full and ready queue will be empty.
- If the scheduler selects **more CPU bound process**, then ready queue will be full and I/O queue will be empty.

2.2.1 Scheduling Queues



- As processes enter the system, they are put into a job queue, which consists of all processes in the system.
- The processes that are residing in main memory and are ready and waiting to execute are kept on a list called the ready queue. This queue is generally stored as a linked list.
- A ready-queue header contains pointers to the first and final PCBs in the list. Each PCB includes a pointer field that points to the next PCB in the ready queue

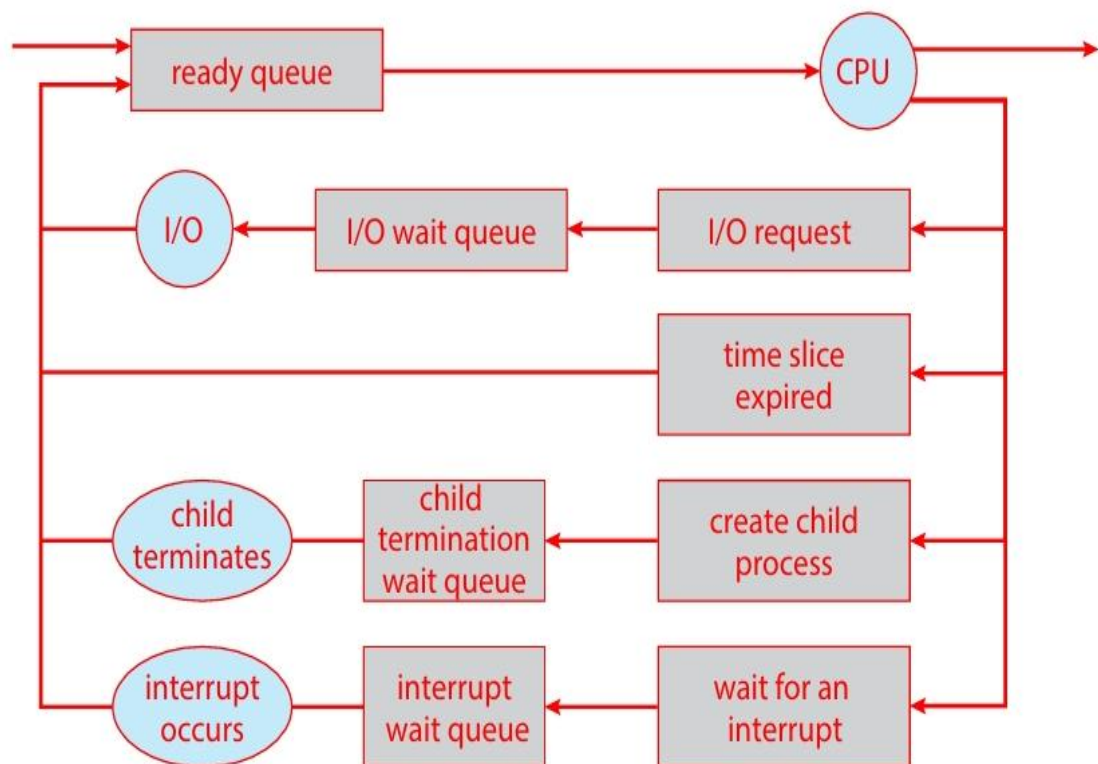


Figure 2.4: Queueing-diagram representation of process scheduling.

- A common representation of process scheduling is a **queueing diagram**. Each rectangular box in the diagram represents a queue. Two types of queues are present: the ready queue and a set of device queues. The circles represent the resources that serve the queues, and the arrows indicate the flow of processes in the system.
- A new process is initially put in the ready queue. It waits in the ready queue until



it is selected for execution and it is given to the CPU. Once the process is allocated the CPU and is executing, one of several events could occur:

- The process could issue an I/O request and then be placed in an I/O wait queue.
- The process could create a new child process and then be placed in a wait queue while it awaits the child termination.
- The process could be removed forcibly from the core, as a result of an interrupt or having its time slice expire, and be put back in the ready queue.

In the first two cases, the process eventually switches from the waiting state to the ready state, and is then put back in the ready queue. A process continues this cycle until it terminates, at which time it is removed from all queues.

2.2.2 CPU Scheduling

A process migrates among the ready queue and various wait queues through out its lifetime. The role of the CPU scheduler is to select from among the processes that are in the ready queue and allocate a CPU core to one of them. The CPU scheduler must select a new process for the CPU frequently. An I/O-bound process may execute for only a few milliseconds before waiting for an I/O request. Although a CPU-bound process will require a CPU core for longer durations, the scheduler is unlikely to grant the core to a process for an extended period. Instead, it is likely designed to forcibly remove the CPU from a process and schedule another process to run. Therefore, the CPU scheduler executes at least once every 100 milliseconds, although typically much more frequently. Some operating systems have an intermediate form of scheduling, known as swapping, whose key idea is that sometimes it can be advantageous to remove a process from memory (and from active contention for the CPU) and thus reduce the degree of multiprogramming. Later, the process can be reintroduced into memory, and its execution can be continued where it left off. This scheme is known as swapping because a process can be “swapped out” from memory to disk, where its current status is saved, and later “swapped in” from disk back to memory, where its status is restored.



2.2.4 Context Switching

- The task of switching a CPU from one process to another process is called context switching. Context-switch times are highly dependent on hardware support (Number of CPU registers).
- Whenever an interrupt occurs (hardware or software interrupt), the state of the currently running process is saved into the PCB and the state of another process is restored from the PCB to the CPU.

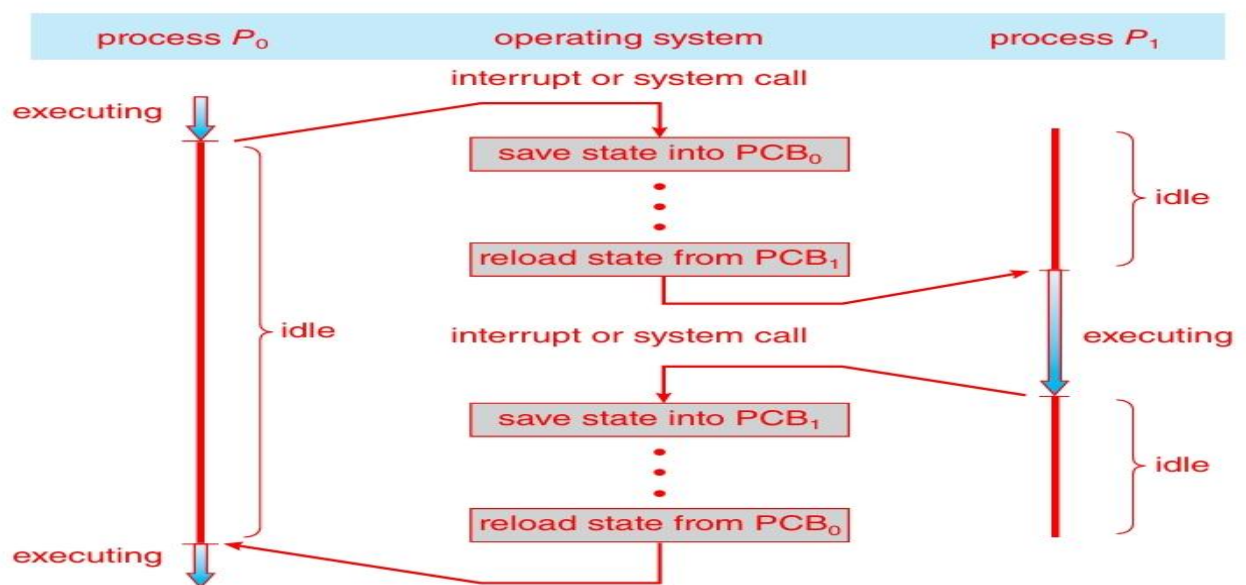


Figure 2.5: Context Switching from two processors

2.3 Schedulers

Schedulers are software which selects an available program to be assigned to CPU.

- A **long-term scheduler or Job scheduler** – selects jobs from the job pool (of secondary memory, disk) and loads them into the memory.

If more processes are submitted, than that can be executed immediately, such processes will be in secondary memory. It runs infrequently, and can take time to select the next process.

- The **short-term scheduler, or CPU Scheduler** – selects job from memory and

assigns the CPU to it. It must select the new process for CPU frequently.

- **The medium-term scheduler** - selects the process in ready queue and reintroduced into the memory.

Processes can be described as either:

Time sharing systems employ a **medium-term scheduler**. It swaps out the process from ready queue and swap in the process to ready queue. When system loads get high, this scheduler will swap one or more processes out of the ready queue for a few seconds, in order to allow smaller faster jobs to finish up quickly and clear the system.

Advantages of medium-term scheduler –

- To remove process from memory and thus reduce the degree of multiprogramming (number of processes in memory).
- To make a proper mix of processes (CPU bound and I/O bound)

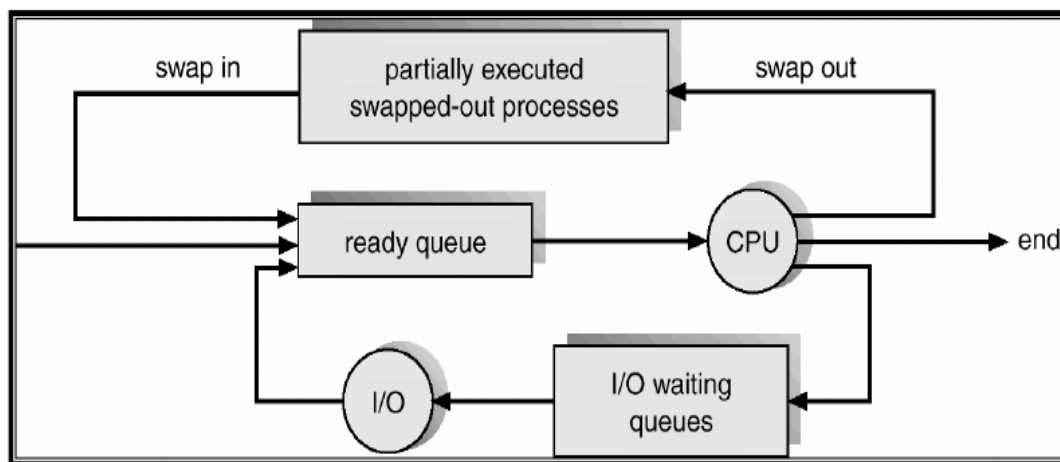


Figure 2.7: Medium-term scheduler

2.4 Operations on Processes

- In most computer systems, **multiple processes run at the same time** (concurrently).
- These processes are **not fixed**; they can be **created** (started) or **deleted** (ended) while the system is running. Because processes come and go dynamically, the operating system needs

to have **special mechanisms** to handle this — to **create new processes** and to **terminate (end) existing ones** properly.

2.4.1 Process Creation:

- A process may create several new processes. The creating process is called a **parent** process, and the new processes are called the **children** of that process. Each of these new processes may in turn create other processes. Every process has a unique process ID.
- On typical Solaris systems, the process at the top of the tree is the '**sched**' process with PID of 0. The '**sched**' process creates several children processes — **init**, **page out** and **fs flush**. Page out and fsflush are responsible for managing memory and file systems. The init process with a PID of 1, serves as a parent process for all user processes.

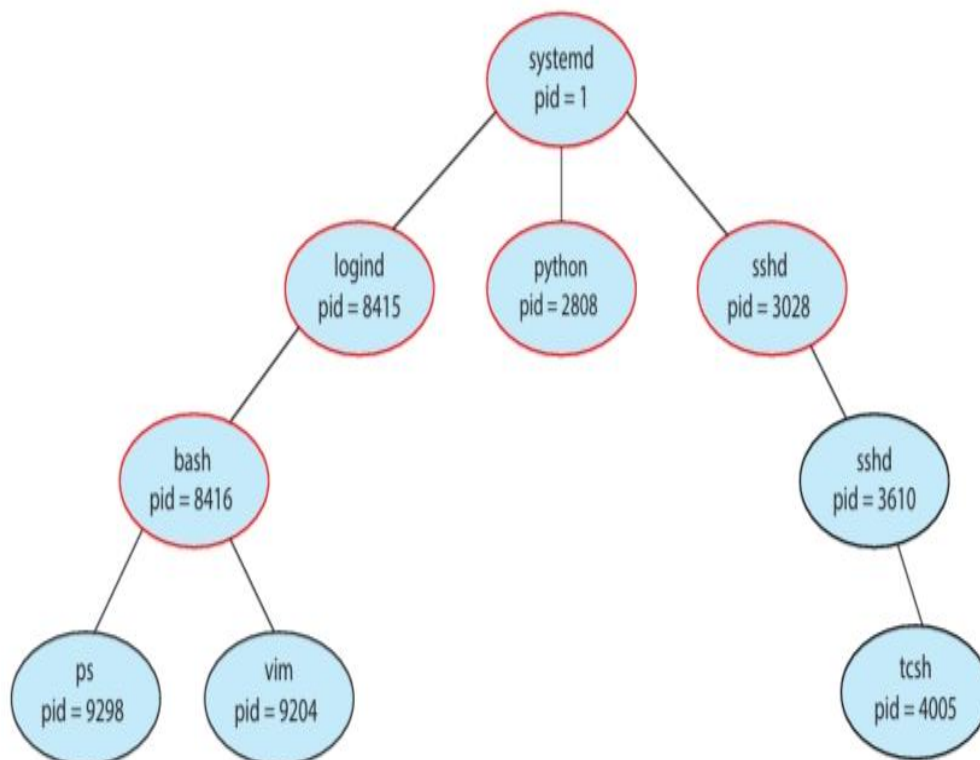


Figure 2.8: A Tree of Processes on typical Solaris System

A process will need certain resources (CPU time, memory, files, I/O devices) to accomplish its task. When a process creates a subprocess, the subprocess may be able to obtain its resources in two ways:

- directly from the operating system
- Subprocess may take the resources of the parent process.
- The resource can be taken from parent in two ways –
 - The parent may have to partition its resources among its children
 - Share the resources among several children.

There are two options for the parent process after creating the child:

- Wait for the child process to terminate and then continue execution. The parent makes a `wait()` system call.
- Run concurrently with the child, continuing to execute without waiting.

Two possibilities for the address space of the child relative to the parent:

- The child may be an exact duplicate of the parent, sharing the same program and data segments in memory. Each will have their own PCB, including program counter, registers, and PID. This is the behavior of the **fork** system call in UNIX.
- The child process may have a new program loaded into its address space, with all new code and data segments. This is the behavior of the **spawn** system calls in Windows.

After a `fork()` system call, one of the two processes typically uses the `exec()` system call to replace the process's memory space with a new program. The `exec()` system call loads a binary file into memory (destroying the memory image of the program containing the `exec()` system call) and starts its execution. In this manner, the two processes are able to communicate and then go their separate



ways. The parent can then create more children; or, if it has nothing else to do while the child runs, it can issue a `wait()` system call to move itself off the ready queue until the termination of the child. Because the call to `exec()` overlays the process's address space with a new program, `exec()` does not return control unless an error occurs.

In UNIX OS, a child process can be created by **`fork()`** system call. The **`fork()`** system call, if successful, returns the PID of the child process to its parents and returns a zero to the child process. If failure, it returns -1 to the parent. Process IDs of current process or its direct parent can be accessed using the `getpid()` and `getppid()` system calls respectively.

The parent waits for the child process to complete with the `wait()` system call. When the child process completes, the parent process resumes and completes its execution.

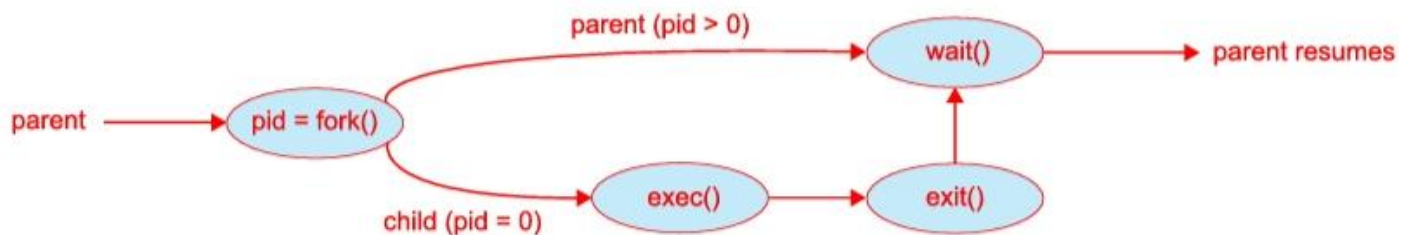


Figure 2.9:Processes Creation

In windows the child process is created using the function **`create process()`**. The `create process()` returns 1, if the child is created and returns 0, if the child is not created.

2.4.2 Process Termination

- A process terminates when it finishes executing its last statement and asks the operating system to delete it, by using the **`exit()`** system call. All of the resources



assigned to the process like memory, open files, and I/O buffers, are deallocated by the operating system.

- A process can cause the termination of another process by using appropriate system call. The parent process can terminate its child processes by knowing of the PID of the child.
- A parent may terminate the execution of children for a variety of reasons, such as:
 - The child has exceeded its usage of the resources, it has been allocated.
 - The task assigned to the child is no longer required.
 - The parent is exiting, and the operating system terminates all the children.

This is called **cascading termination**.

2.5 Inter process Communication

Inter process Communication- Processes executing may be either co-operative or independent processes.

- **Independent Processes** – processes that cannot affect other processes or be affected by other processes executing in the system.
- **Cooperating Processes** – processes that can affect other processes or be affected by other processes executing in the system.

Co-operation among processes are allowed for following reasons –

- **Information Sharing** - There may be several processes which need to access the same file. So the information must be accessible at the same time to all users.

Computation speedup - Often a solution to a problem can be solved faster if the problem can be broken down into sub-tasks, which are solved simultaneously (particularly when multiple processors are involved.)

- **Modularity** - A system can be divided into cooperating modules and executed by sending information among one another.
- **Convenience** - Even a single user can work on multiple tasks by information sharing.

Cooperating processes require some type of inter-process communication. This is allowed by two models:

1. Shared Memory systems: In the shared-memory model, a region of memory that is shared by the cooperating processes is established. Processes can then exchange information by reading and writing data to the shared region.
2. Message passing systems: In the message-passing model, communication takes place by means of messages exchanged between the cooperating processes.

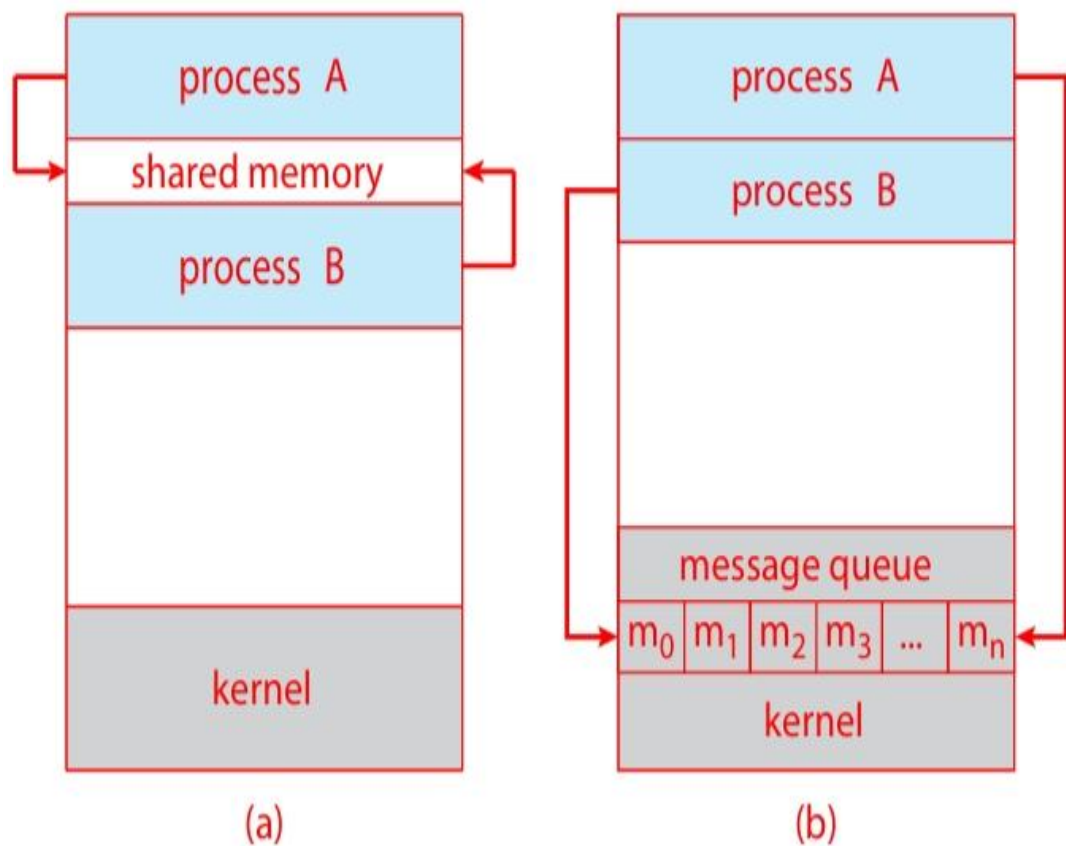


Figure 2.10: Communication model (a) Shared Memory Message Passing (b) Message Passing

Message passing is useful for exchanging smaller amounts of data, because no conflicts need be avoided.

Message passing is also easier to implement in a distributed system than shared memory.

Shared memory can be faster than message passing, since message-passing systems are



typically implemented using system calls and thus require the more time-consuming task of kernel intervention.

Sl No	Shared Memory	Message passing
1.	A region of memory is shared by communicating processes, into which the information is written and read	Message exchange is done among the processes by using objects.
2.	Useful for sending large block of Data	Useful for sending small data.
3.	System call is used only to create shared memory	System call is used during every read and write operation.
4.	Message is sent faster, as there are no system calls	Message is communicated slowly.

- **Shared Memory** is faster once it is set up, because no system calls are required and access occurs at normal memory speeds. Shared memory is generally preferable when large amounts of information must be shared quickly on the same computer.
- **Message Passing** requires system calls for every message transfer, and is therefore slower, but it is simpler to set up and works well across multiple computers. Message passing is generally preferable when the amount and/or frequency of data transfers is small.



2.5.1 IPC in Shared-Memory Systems

- A region of shared-memory is created within the address space of a process, which needs to communicate. Other process that needs to communicate uses this shared memory.
- The form of data and position of creating shared memory area is decided by the process. Generally, a few messages must be passed back and forth between the cooperating processes first in order to set up and coordinate the shared memory access.
- The process should take care that the two processes will not write the data to the shared memory at the same time.

Producer-Consumer Example Using Shared Memory

- This is a classic example, in which one process is producing data and another process is consuming the data.
- The data is passed via an intermediary buffer (shared memory). The producer puts the data to the buffer and the consumer takes out the data from the buffer. A producer can produce one item while the consumer is consuming another item. The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced. In this situation, the consumer must wait until an item is produced.
- There are two types of buffers into which information can be put –
 - Unbounded buffer
 - Bounded buffer
- **With Unbounded buffer**, there is no limit on the size of the buffer, and so on the data produced by producer. But the consumer may have to wait for new items.

- **With bounded-buffer** – As the buffer size is fixed. The producer has to wait if the buffer is full and the consumer has to wait if the buffer is empty.

2.5.2 IPC in Message Passing

A mechanism to allow process communication without sharing address space. It is used in distributed systems.

- Message passing systems uses system calls for "send message" and "receive message".
- A communication link must be established between the cooperating processes before messages can be sent.
- There are three methods of creating the link between the sender and the receiver-
 - Direct or indirect communication (naming)
 - Synchronous or asynchronous communication (Synchronization)
 - Automatic or explicit buffering.

Naming

Processes that want to communicate must have a way to refer to each other. They can use either direct or indirect communication.

- **Direct communication** the sender and receiver must explicitly know each other's name. The syntax for send() and receive() functions are as follows-
send (*P, message*) – send a message to process P
receive(*Q, message*) – receive a message from process Q

Properties of communication link:

- A link is established automatically between every pair of processes that wants to communicate. The processes need to know only each other's identity to communicate.

- A link is associated with exactly one pair of communicating processes between each pair, there exists exactly one link.

Types of addressing in direct communication –

Symmetric addressing – the above-described communication is symmetric communication. Here both the sender and the receiver processes have to name each other to communicate.

Asymmetric addressing – Here only the sender's name is mentioned, but the receiving data can be from any system. **send (P, message)** --- Send a message to process **receive (id, message)**. Receive a message from any process

Disadvantages of direct communication – any changes in the identifier of a process, may have to change the identifier in the whole system (sender and receiver), where the messages are sent and received.

- **Indirect communication** uses shared mailboxes, or ports.

A mailbox or port is used to send and receive messages. Mailbox is an object into which messages can be sent and received. It has a unique ID. Using this identifier messages are sent and received.

Two processes can communicate only if they have a shared mailbox. The send and receive functions are –

- **send (A, message)** – send a message to mailbox A
- **receive (A, message)** – receive a message from mailbox A

Properties of communication link:

A link is established between a pair of processes only if they have a shared mailbox.

- A link may be associated with more than two processes
- Between each pair of communicating processes, there may be any number of links, each link is associated with one mailbox.



- A mail box can be owned by the operating system. It must take steps to –
 - create a new mailbox
 - send and receive messages from mailbox
 - delete mailboxes.

- **Synchronization**

The send and receive messages can be implemented as either **blocking** or **non-blocking**.

- ☐ **Blocking (synchronous) send** - sending process is blocked (waits) until the message is received by receiving process or the mailbox.
- ☐ **Non-blocking (asynchronous) send** - sends the message and continues (does not wait)
- ☐ **Blocking (synchronous) receive** - The receiving process is blocked until a message is available
- ☐ **Non-blocking (asynchronous) receive** - receives the message without block. The received message may be a valid message or null.

- **Buffering**

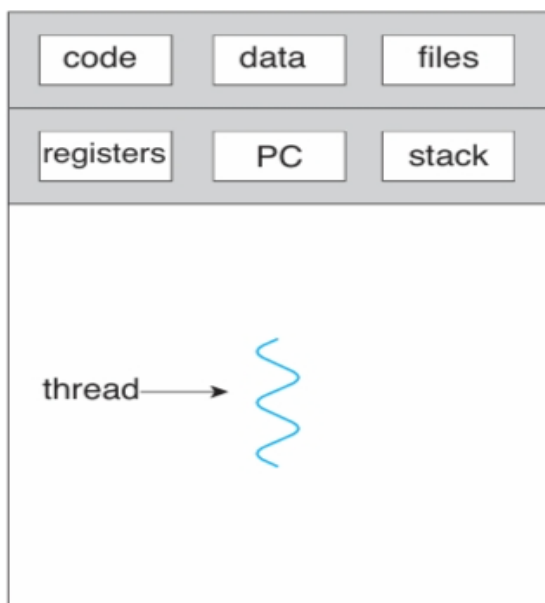
When messages are passed, a temporary queue is created. Such queue can be of three capacities:

- ☐ **Zero capacity** – The buffer size is zero (buffer does not exist). Messages are not stored in the queue. The senders must block until receivers accept the messages.
- ☐ **Bounded capacity**- The queue is of fixed size(n). Senders must block if the queue is full. After sending 'n' bytes the sender is blocked.
- ☐ **Unbounded capacity** - The queue is of infinite capacity. The sender never blocks.

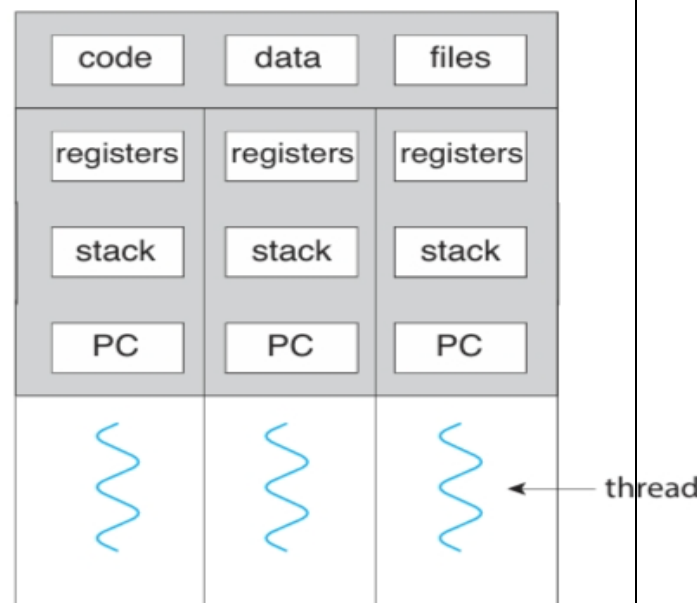
Threads & Concurrency

3.1 Overview

- A thread is a basic unit of CPU utilization.
- It consists of
 - thread ID
 - PC
 - register-set and
 - stack.
- It shares with other threads belonging to the same process its code-section & data-section.
- A traditional (or heavy weight) process has a single thread of control.
- If a process has multiple threads of control, it can perform more than one task at a time. such a process is called **multithreaded process**



single-threaded process



multithreaded process

Figure 3.1: Single-threaded and multithreaded processes

3.2 Motivation for Multithreaded Programming

1. The software-packages that run on modern PCs are multi threaded. An application is implemented as a separate process with several threads of control. For ex: A word processor may have
 - first thread for displaying graphics
 - second thread for responding to key strokes and
 - Third thread for performing grammar checking.
2. In some situations, a single application may be required to perform several similar tasks. For ex: A web-server may create a separate thread for each client requests. This allows the server to service several concurrent requests.
3. RPC servers are multithreaded.
 - When a server receives a message, it services the message using separate concurrent threads.
4. Most OS kernels are multithreaded;
 - Several threads operate in kernel, and each thread performs a specific task, such as managing devices or interrupt handling.

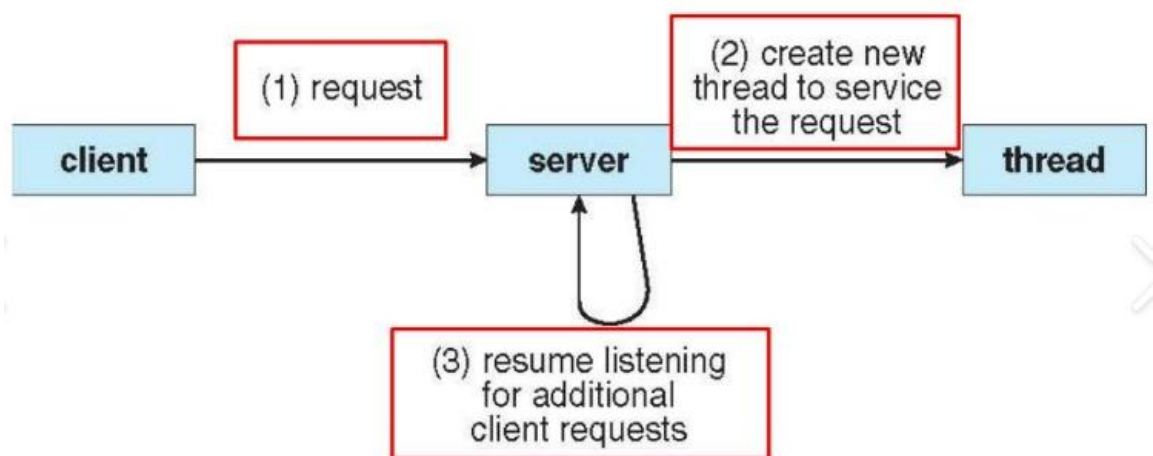


Figure 3.2 :Multithread server architecture

3.3 Benefits of Multithreaded Programming

- **Responsiveness** A program may be allowed to continue running even if part of it is blocked. Thus, increasing responsiveness to the user.
- **Resource Sharing** By default, threads share the memory (and resources) of the process to which they belong. Thus, an application is allowed to have several different threads of activity within the same address-space.
- **Economy** Allocating memory and resources for process-creation is costly. Thus, it is more economical to create and context-switch threads.
- **Utilization of Multiprocessor Architectures** In a multiprocessor architecture, threads may be running in parallel on different processors. Thus, parallelism will be increased.

3.4 Multicore Programming`

A modern step where multiple CPU cores are placed on a single chip. Each core acts like an independent CPU. This increases performance while reducing physical space and power consumption compared to having multiple separate CPUs.

- ❑ **Single-core CPU** can execute only **one thread at a time**.
- ❑ Multiple threads appear to run simultaneously by rapidly switching between them.
- ❑ This rapid switching is called **context switching**.
- ❑ Each thread runs for a short time slice before the CPU switches to the next thread.
- ❑ This creates the illusion of parallelism but actually is **concurrency without parallelism**.

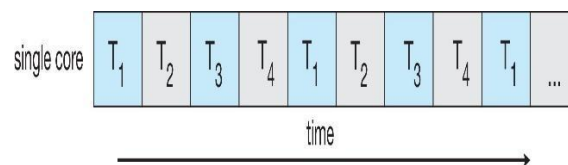


Figure 3.3: Concurrent execution on single-core

- ❑ system: The CPU executes **Thread 1** for a small time slice (quantum).
- ❑ Then switches to **Thread 2** and executes it for another slice.
- ❑ Then switches to **Thread 3**.

□ This cycle repeats quickly.

Multicore or multiprocessor systems putting pressure on programmers,

3.5 Programming challenges:

1. **Identifying tasks:** This involves examining applications to find areas that can be divided into separate, concurrent tasks. Ideally, tasks are independent of one another and thus can run in parallel on individual cores.

2. **Balance:** While identifying tasks that can run in parallel, programmers must also ensure that the tasks perform equal work of equal value. In some instances, a certain task may not contribute as much value to the overall process as other tasks. Using a separate execution core to run that task may not be worth the cost.

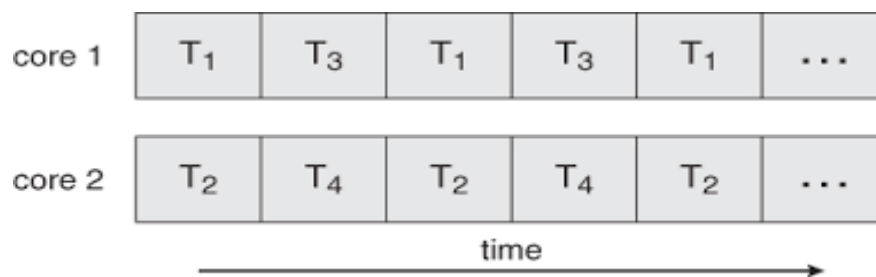


Figure 3.4: Parallelism on a multi-core system:

3. **Data splitting:** Just as applications are divided into separate tasks, the data accessed and manipulated by the tasks must be divided to run on separate cores.

4. **Data dependency:** The data accessed by the tasks must be examined for dependencies between two or more tasks. When one task depends on data from another, programmers must ensure that the execution of the tasks is synchronized to accommodate the data dependency.

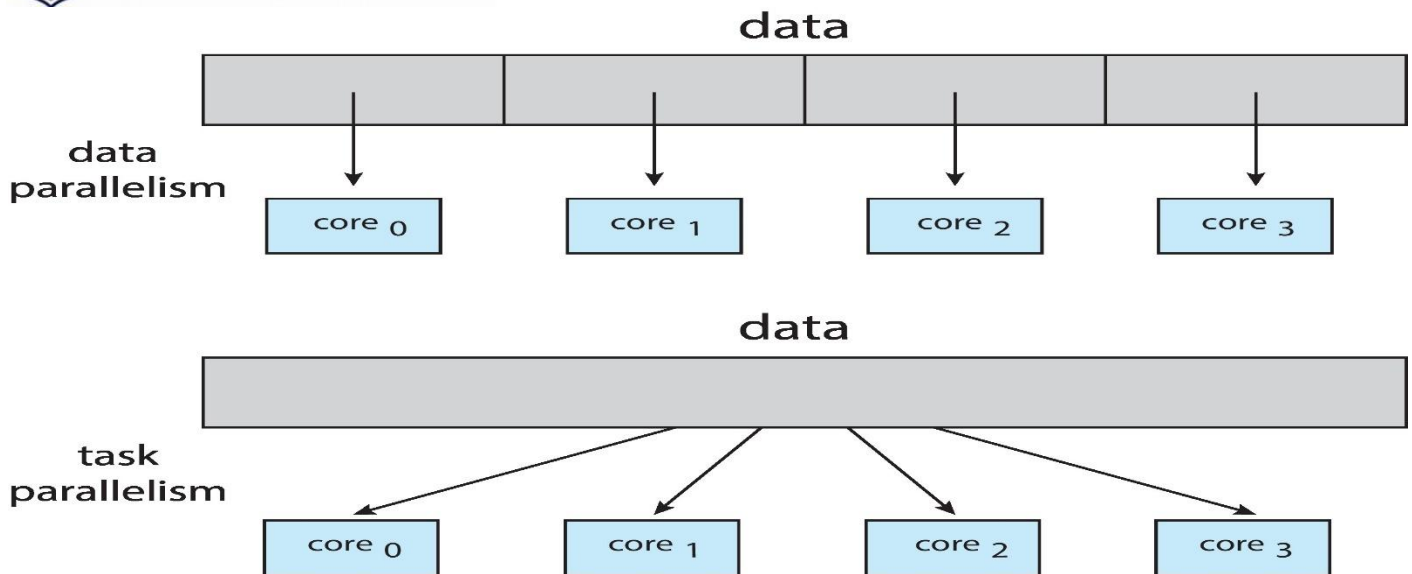


Figure 3.5: Data and task parallelism.

5. Testing and debugging: When a program is running in parallel on multiple cores, many different execution paths are possible. Testing and debugging such concurrent programs is inherently more difficult than testing and debugging single-threaded applications.

Types of parallelism:

Data Parallelism:

Data parallelism focuses on distributing subsets of the same data across multiple computing cores and performing the same operation on each core.

Example: summing the contents of an array of size N . On a single-core system, one thread would simply sum the elements $[0] \dots [N-1]$. On a dual-core system, however, thread A, running on core 0, could sum the elements $[0] \dots [N/2-1]$ while thread B, running on core 1, could sum the elements $[N/2] \dots [N-1]$. The two threads would be running in parallel on separate computing cores.

Task parallelism : It involves distributing not data but tasks (threads) across multiple computing cores. Each thread is performing a unique operation. Different threads may be operating on the same data, or they may be operating on different data. Consider again our example above. In contrast to that situation, an example of task parallelism might involve two threads, each performing a unique statistical operation on the array of elements. The threads again are operating in parallel on separate computing cores, but each is performing a unique operation.

3.6 MULTITHREADING MODELS

- Support for threads may be provided at either
 1. The user level, for **user threads** or
 2. By the kernel, for **kernel threads**.
- User-threads are supported above the kernel and are managed without kernel support. Kernel- threads are supported and managed directly by the OS.
- Three ways of establishing relationship between user-threads & kernel-threads:
 1. Many-to-one model
 2. One-to-one model and
 3. Many-to-many model.

➤ **Many-to-One Model**

- Many user-level threads are mapped to one kernel thread.

Advantages:

- Thread management is done by the thread library in user space, so it is efficient.

Disadvantages:

- The entire process will block if a thread makes a blocking system-call.
- Multiple threads are unable to run in parallel on multiprocessors.

- For example:
 - Solaris green threads
 - GNU portable threads.

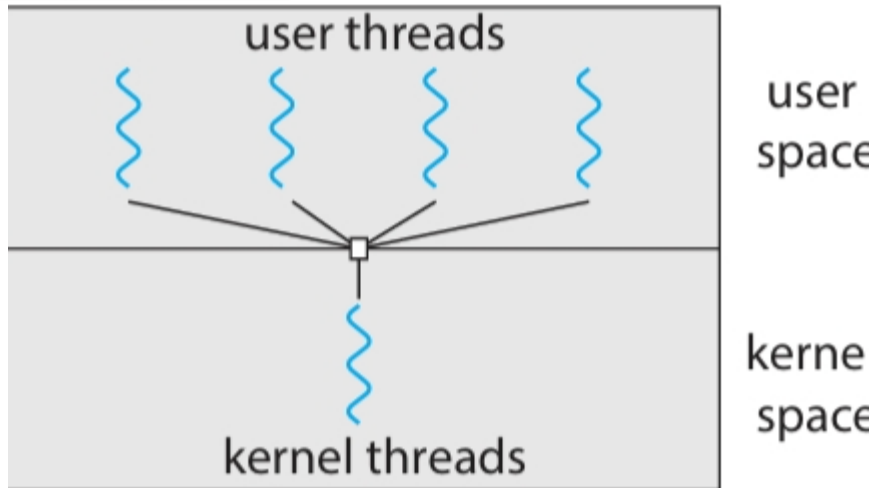


Figure 3.6: Many-to-one model

- Each user thread is mapped to a kernel thread.

Advantages:

- It provides more concurrency by allowing another thread to run when a thread makes a blocking system-call.
- Multiple threads can run in parallel on multiprocessors.

Disadvantage:

- Creating a user thread requires creating the corresponding kernel thread.
- For example:
 - Windows NT/XP/2000, Linux

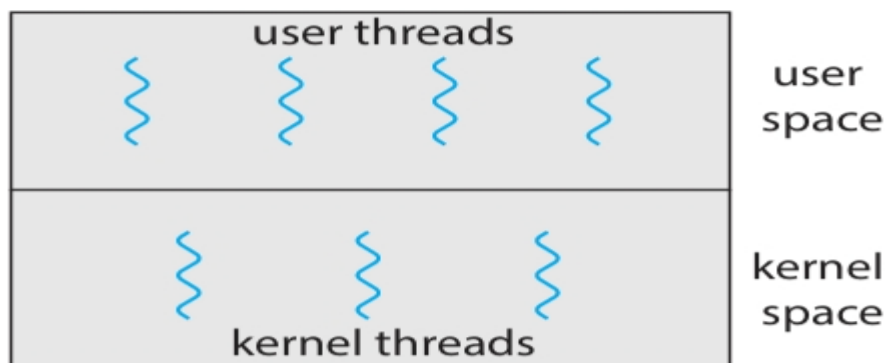


Figure 3.7: one-to-one model

➤ **Many-to-Many Model**

- Many user-level threads are multiplexed to a smaller number of kernel threads.

Advantages:

- Developers can create as many user threads as necessary
- The kernel threads can run in parallel on a multiprocessor.
- When a thread performs a blocking system-call, kernel can schedule another thread for execution.

➤ **Two Level Model**

- A variation on the many-to-many model is the two level-model
- Similar to M:N, except that it allows a user thread to be bound to kernel thread.
- For example:
 - HP-UX
 - Tru64 UNIX

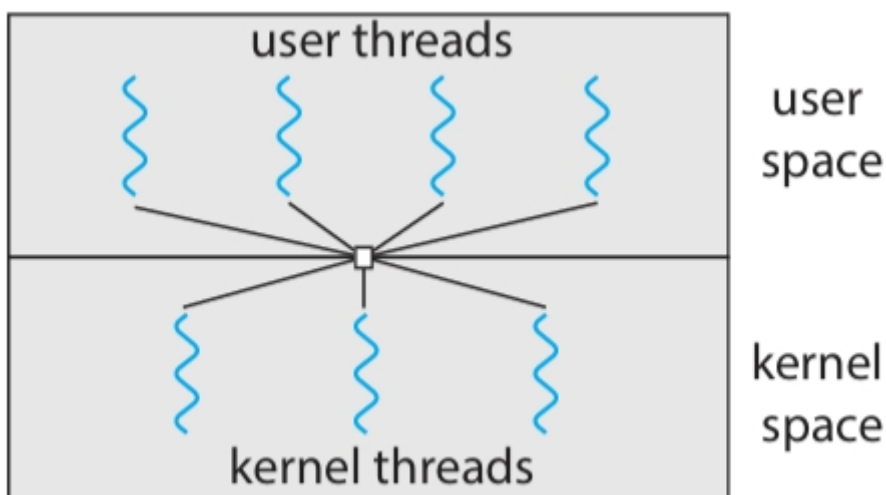


Figure 3.8: Many-to-many model

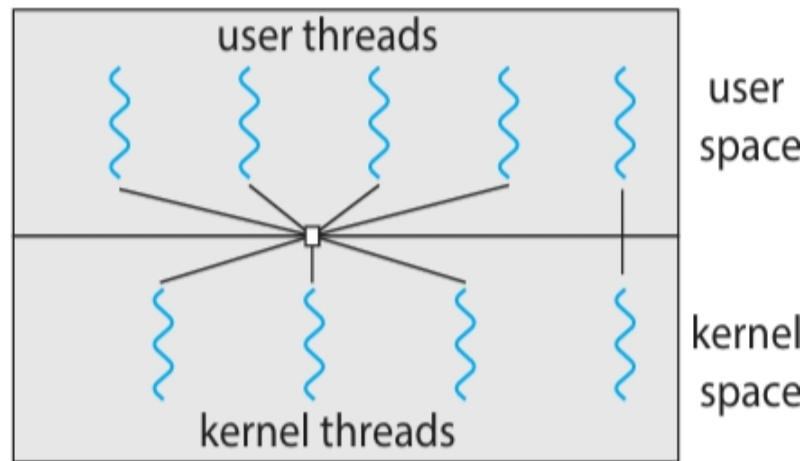


Figure 3.9: Two-level model